

Hitos generativos y arquitectura transformer

MMIA 6013 — IA Generativa y Agentes · Sesión 02 · 15 de septiembre de 2026

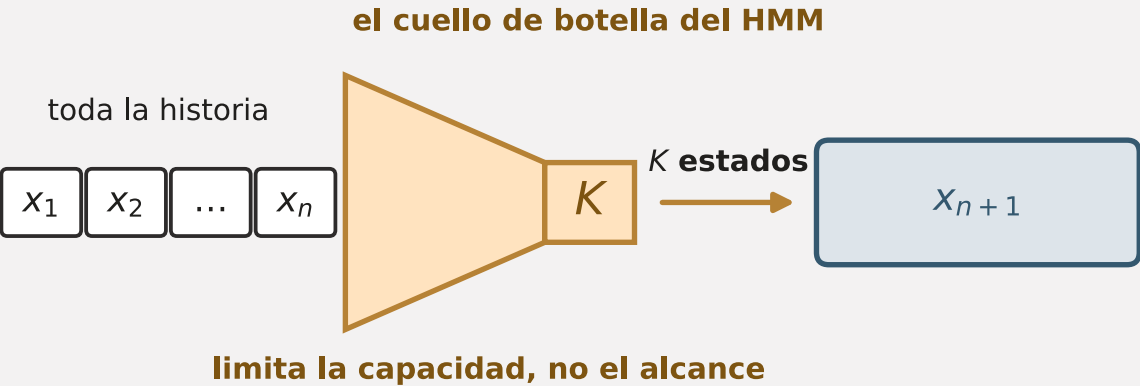
Daniel Andrés Riofrío Almeida

Dónde quedamos: cuatro hitos y un cuello de botella

- La mezcla de gaussianas (GMM) factoriza $P(X)$ con un latente discreto; Naive Bayes no tiene latente, porque la clase se observa al entrenar. Ninguno de los dos modela secuencia
- El modelo oculto de Markov (HMM) sí es secuencial: $p(x_{n+1} \mid x_1, \dots, x_n)$ depende de todas las observaciones anteriores
- Lo limitado es la capacidad, no el alcance: toda esa historia tiene que caber en K estados discretos
- El autocodificador variacional (VAE) es el hito que hizo continuo el latente, y es la lectura primaria asignada a hoy
- Bishop ya apuntaba la salida en 2006: usar un modelo paramétrico, ‘como por ejemplo una red neuronal’, para $p(x_n \mid x_{n-M}, \dots, x_{n-1})$
- Lo que falta, y es el contenido de hoy, es qué red

El HMM alcanza toda la historia; lo que le falta es capacidad

hito	variable latente	limitación que hereda
Naive Bayes	ninguna: Y se observa	no modela secuencia
GMM	componente discreta	no modela secuencia
HMM	estado discreto z_n	toda la historia cabe en K estados
VAE	z continuo	posterior $p(z \mid x)$ intratable



EM: el método que el VAE va a romper

- Con latente, maximizar $\ln p(X | \theta)$ es duro: el logaritmo cae sobre la suma de la marginal y no se despeja
- La receta son cuatro pasos: inicializar θ ; **paso E**, evaluar la posterior del latente con θ fijo; **paso M**, reestimar θ con esa posterior fija; medir la log-verosimilitud y repetir
- En el GMM, el paso E son las responsabilidades $\gamma(z_{nk})$ y el paso M reestima μ_k , Σ_k y π_k usándolas como pesos
- En general el paso M maximiza $Q(\theta, \theta^{\text{old}}) = \sum_Z p(Z | X, \theta^{\text{old}}) \ln p(X, Z | \theta)$: el logaritmo pasa a actuar sobre la conjunta, y por eso sí es tratable
- La garantía: cada ciclo **no baja** la log-verosimilitud, salvo que ya esté en un máximo local. No promete el máximo global
- Bishop es explícito en las dos trampas: hay múltiples máximos locales y hay singularidades, y por eso suele inicializarse con K-medias

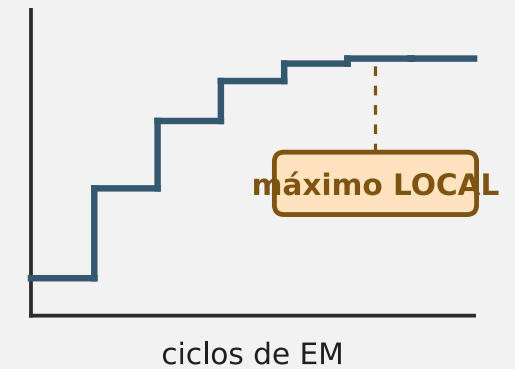
EM alterna: cada paso congela una mitad

paso E
con θ congelado,
calcula $p(z | x)$

paso M
con $p(z | x)$ congelada,
reestima θ

**se repite hasta que la
log-verosimilitud deja de subir**

log-verosimilitud

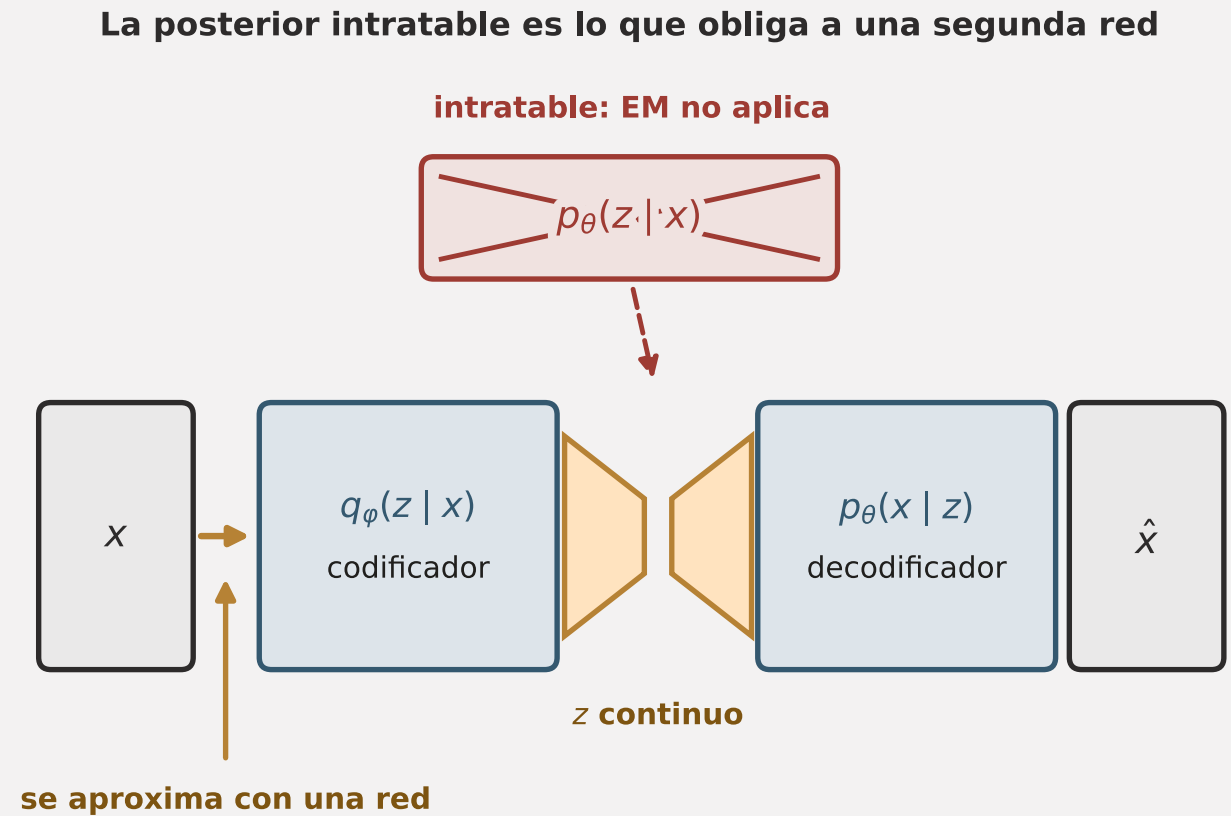


Ningún ciclo empeora el ajuste. Eso es todo lo que promete:

Bishop advierte que hay máximos locales y singularidades.

VAE: cuando el paso E deja de ser computable

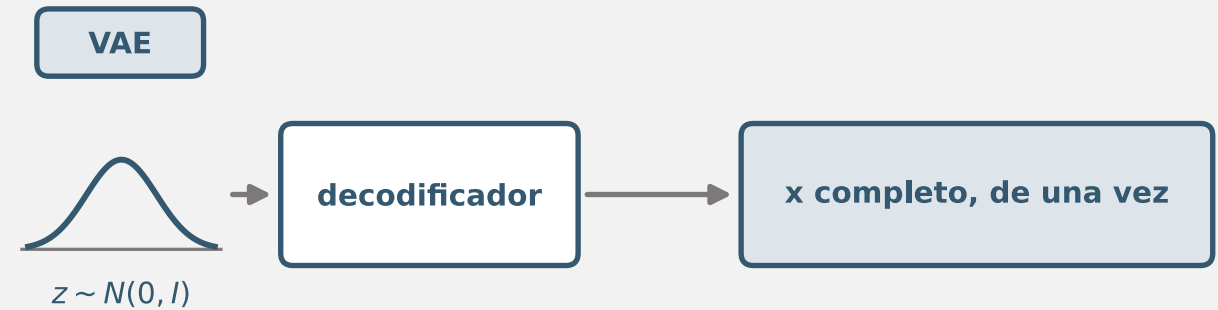
- Escenario en dos pasos: se muestrea z de una prior y se genera x de $p_\theta(x | z)$. Es muestreo ancestral, ahora con z continuo
- La verosimilitud marginal $p_\theta(x) = \int p_\theta(z) p_\theta(x | z) dz$ es intratable, y con ella la posterior verdadera $p_\theta(z | x)$
- El algoritmo de esperanza-maximización (EM) deja de servir; literal del paper: ‘de modo que el algoritmo EM no puede usarse’. En el GMM el paso E era Bayes exacto y en forma cerrada
- La salida es aproximar esa posterior con una segunda red: el codificador probabilístico $q_\phi(z | x)$
- Se maximiza una cota inferior variacional (ELBO): reconstrucción menos una divergencia de Kullback-Leibler (KL) contra la prior $N(0, I)$
- Truco de reparametrización: $z = \mu + \sigma \cdot \varepsilon$ con $\varepsilon \sim N(0, I)$, para poder derivar a través del muestreo



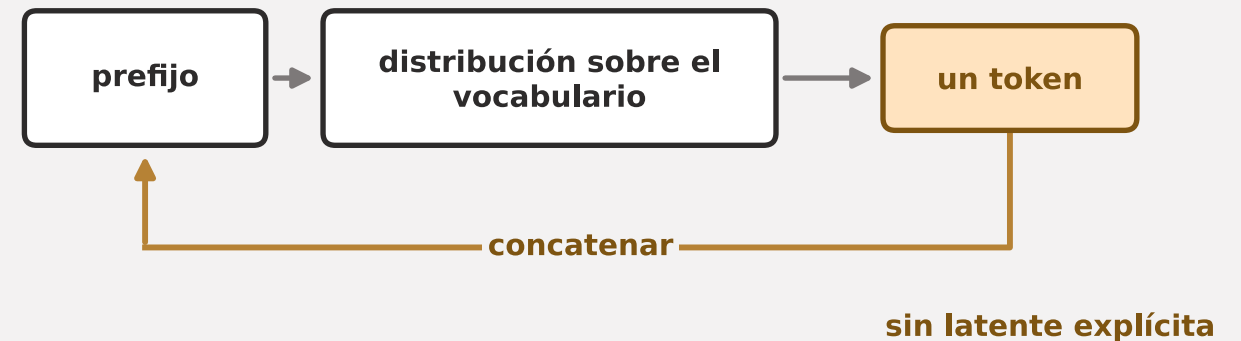
Con latente explícito y sin él

- VAE: $P(X) = \int p(z) p(x | z) dz$ — marginaliza un latente continuo
- Modelo grande de lenguaje (LLM) autorregresivo: $P(X) = \prod_{t=1}^n P(x_t | x_{<t})$ — regla del producto, sin latente
- Generar con el VAE: muestrear $z \sim N(0, I)$ y decodificar todo x de una vez
- Generar con el LLM: muestrear un token, concatenarlo al contexto y repetir
- El precio del VAE es la posterior intratable; el del LLM, que el contexto crece y la atención cuesta cuadrático
- Lo que el VAE lega: la idea de comprimir y descomprimir, y el nombre codificador-decodificador

Ser generativo no implica tener una latente



LLM autorregresivo



El transformer

02

Prescindir de la recurrencia

- Vaswani §1: la naturaleza secuencial de las redes neuronales recurrentes (RNN) ‘impide la paralelización’ dentro de cada ejemplo de entrenamiento
- El estado oculto de tamaño fijo de la RNN es el mismo cuello de botella del HMM con otro vocabulario
- La atención es de 2014 y se usaba **con** recurrencia; lo que 2017 propone es prescindir de ella y apoyarse enteramente en atención
- Tabla 1: la auto-atención cuesta $O(n^2 \cdot d)$ por capa; la recurrente, $O(n \cdot d^2)$
- A cambio, las operaciones secuenciales pasan de $O(n)$ a $O(1)$ y el camino entre dos posiciones, de $O(n)$ a $O(1)$
- Se cambia profundidad secuencial por cómputo cuadrático. La atención no elimina el costo, lo mueve

La atención no elimina el costo: lo mueve			
capa	complejidad por capa	operaciones secuenciales	camino máximo entre posiciones
auto-atención	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
recurrente	$O(n \cdot d^2)$	$O(n)$	$O(n)$
convolucional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$

n = longitud en tokens, d = dimensión de la representación

sube el cómputo por capa

$O(n \cdot d^2) \rightarrow O(n^2 \cdot d)$

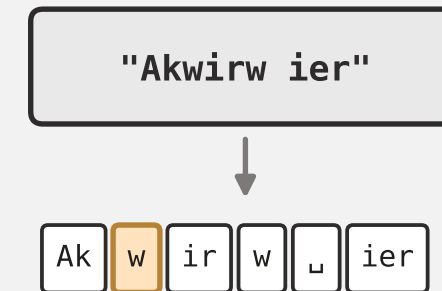
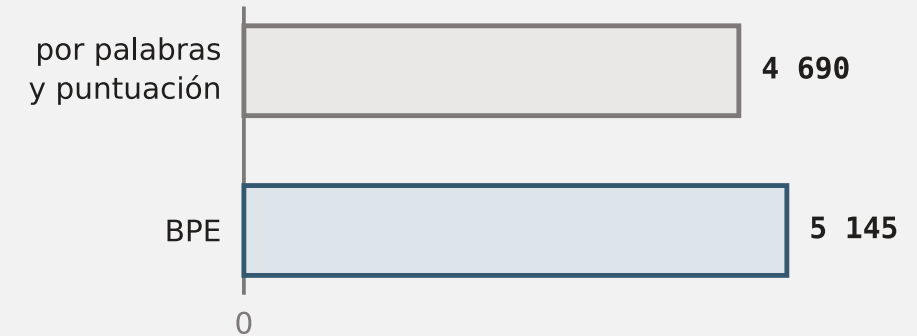
baja la profundidad secuencial

$O(n) \rightarrow O(1)$

Un token no es una palabra

- El pipeline de entrada: texto → tokens → IDs de token → representaciones vectoriales (embeddings), y encima la señal de posición
- El vocabulario de GPT-2 y GPT-3 tiene 50 257 tokens; el byte pair encoding (BPE) parte de caracteres y fusiona iterativamente los pares frecuentes
- BPE no necesita un token para lo desconocido: parte 'Akwirw ier' en seis piezas
- El mismo cuento de 20 479 caracteres da 4 690 tokens contando palabras y puntuación, y 5 145 con BPE
- Mismo texto, dos conteos: el número depende del tokenizador, y es exactamente lo que se cobra
- La capa de embedding es una búsqueda por identificador (ID) sobre una matriz de pesos entrenable, no una tabla fija. Cómo se aprende ese espacio es la sesión 06

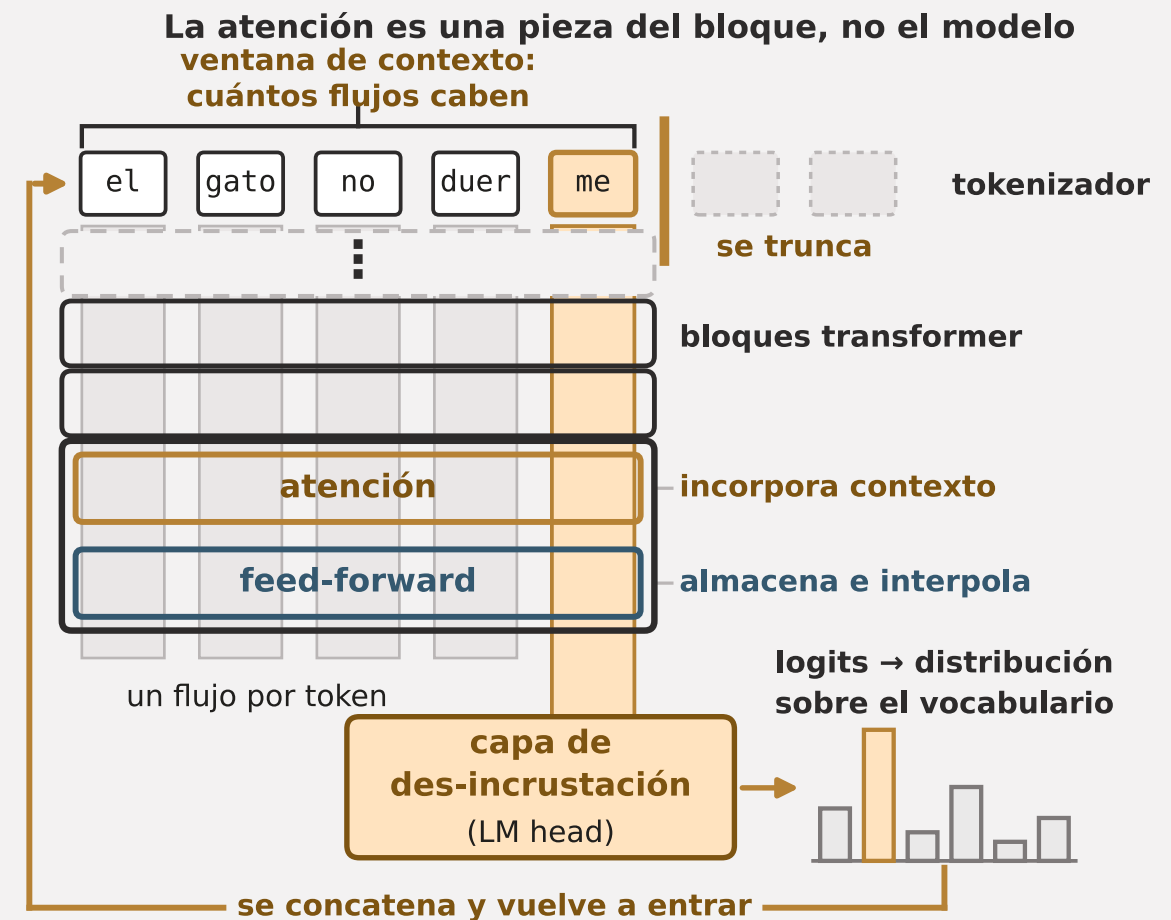
El mismo texto, dos conteos distintos



seis piezas: BPE no necesita un token para lo desconocido

Tokenizador, bloques y des-incrustación: dónde cabe lo que cabe

- El modelo son tres piezas: tokenizador, pila de bloques transformer y capa de des-incrustación (LM head), que proyecta al vocabulario y da los logits; el softmax los vuelve distribución
- Un paso de generación es un *forward pass*: entra el texto, sale un token, el token se concatena al texto de entrada y todo vuelve a entrar. Eso es ser autorregresivo
- Cada bloque tiene dos componentes: la capa de atención, que incorpora información de otras posiciones, y una red feed-forward, donde vive la mayor parte de lo memorizado
- Cada token recorre su propio flujo de cómputo; los flujos solo se comunican en la atención, y al generar solo el del último alimenta la capa de des-incrustación
- La ventana de contexto es cuántos flujos caben a la vez: lo que excede se trunca, sin desbordamiento elegante. Cifra histórica citable: 1 024 tokens en el GPT-2 más pequeño
- No es memoria: el recorrido vuelve a empezar en 'texto' en cada llamada. Ninguna ventana de un modelo vigente se proyecta hoy: fuentes/modelos/modelos-2026-1.json no tiene ese campo



Atención: query, key, value

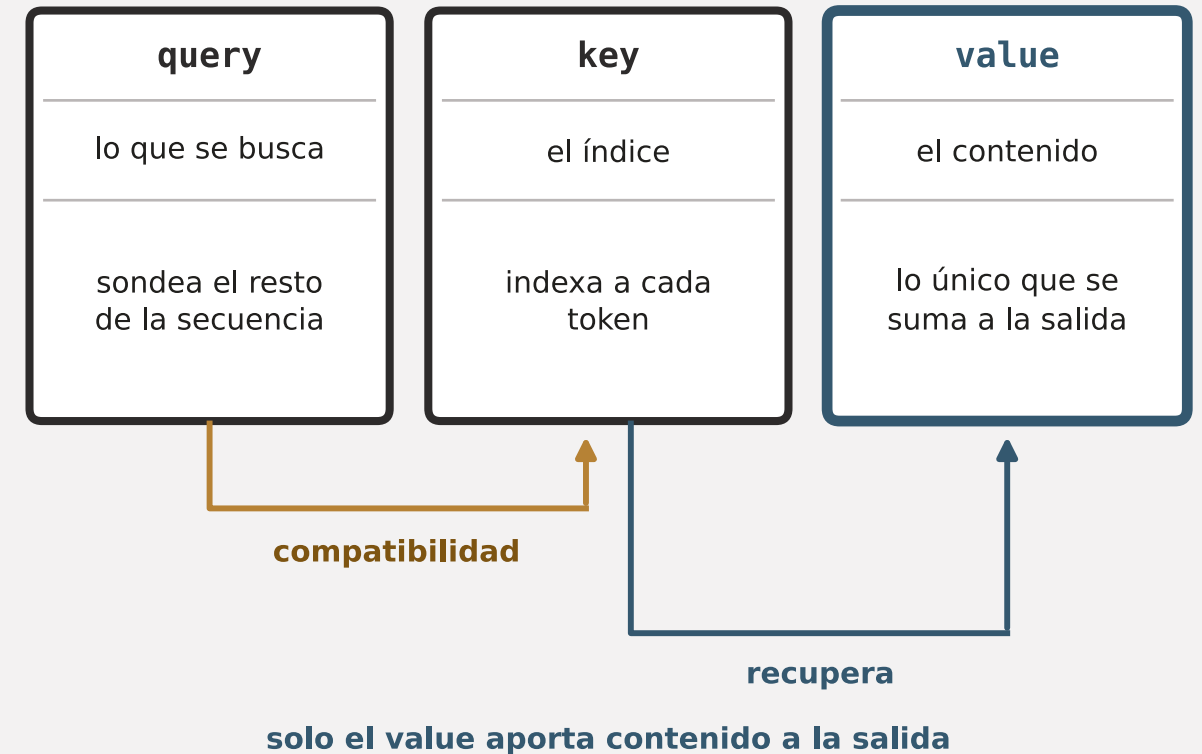
02

La atención es una suma ponderada

- Vaswani §3.2: mapear una query y un conjunto de pares key-value a una salida; query, keys, values y salida son todos vectores
- La salida es una suma ponderada de los values; el peso de cada value sale de una función de compatibilidad entre la query y su key
- Auto-atención: las tres cosas salen de una misma secuencia, no de dos secuencias distintas
- El vocabulario viene de recuperación de información y bases de datos: la query sondea, la key indexa, el value es el contenido
- Dicho como mecanismo y no como metáfora: paso 1, puntuar cuán relevante es cada posición anterior; paso 2, combinar sus values con esos puntajes en un solo vector de salida
- Es una analogía de procedencia del vocabulario, no una descripción de lo que el modelo entiende

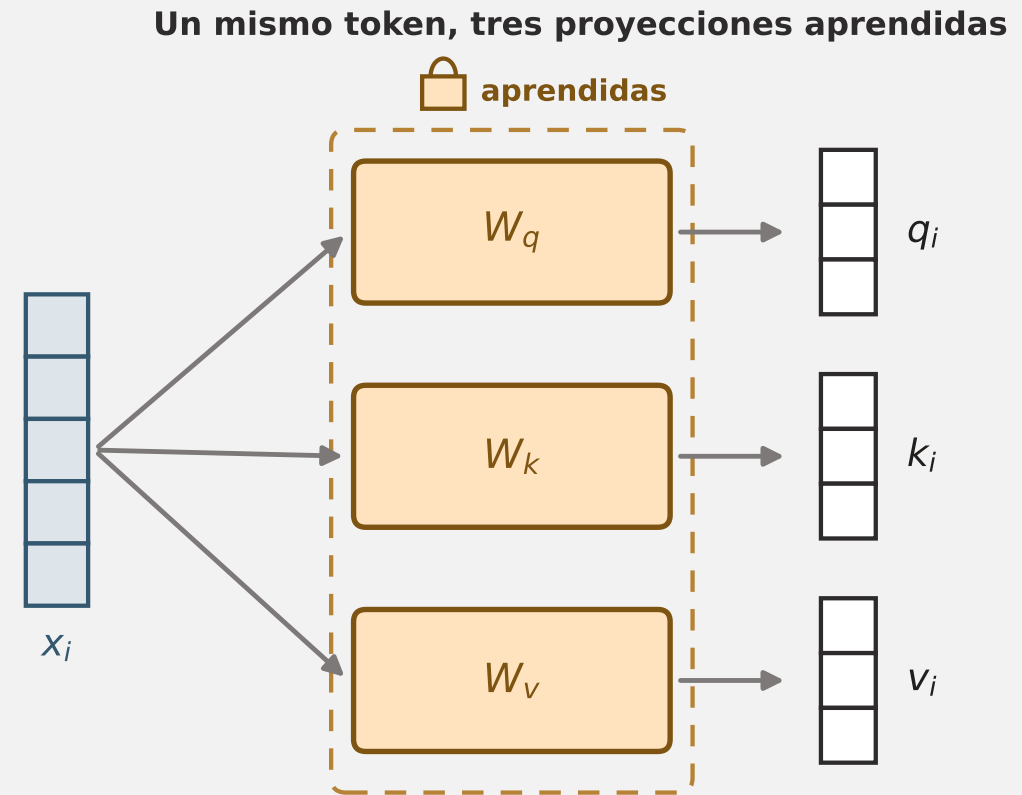
Query sondea, key indexa, value aporta el contenido

arriba, el papel en una base de datos · abajo, en el transformer



Q, K y V salen de tres matrices entrenables

- W_q, W_k y W_v proyectan cada token ya vectorizado en su query, su key y su value
- $q_i = x_i W_q, k_i = x_i W_k, v_i = x_i W_v$: el mismo token proyectado a tres espacios distintos, uno por cada papel
- Son matrices de forma $(d_{\text{in}}, d_{\text{out}})$; con **una sola cabeza** las dos dimensiones suelen coincidir. Con h cabezas no: $d_k = d_{\text{model}}/h$, y eso es la diapositiva 17
- Sus formas, tal como las formaliza Vaswani: W^Q y W^K son $d_{\text{model}} \times d_k$; W^V es $d_{\text{model}} \times d_v$
- d_{model} es la dimensión que comparten las representaciones y las sub-capas del modelo
- Sin proyecciones distintas, el score de un token consigo mismo domina por construcción, no por semántica



tres vistas del mismo token, no tres cosas del texto

Los pesos de atención de esta frase, ¿son parámetros del modelo?

- A. Sí: se aprendieron durante el entrenamiento y quedaron guardados con los demás pesos
- B. No: se recalculan para cada entrada; lo aprendido son W_q , W_k y W_v
- C. Depende: en entrenamiento son dinámicos y en inferencia quedan fijos

Dos cosas que se llaman peso

parámetros de peso

W_q

W_k

W_v

salen del entrenamiento

¿cambia con la entrada?

¿se guarda con el modelo?

pesos de atención

	t1	t2	t3
t1			
t2			
t3			

la que sale del softmax para esta frase

¿cambia con la entrada?

¿se guarda con el modelo?

Fuente: Raschka (2024), cap. 3, p. 66 — el recuadro se nombra en voz alta después de recoger las respuestas

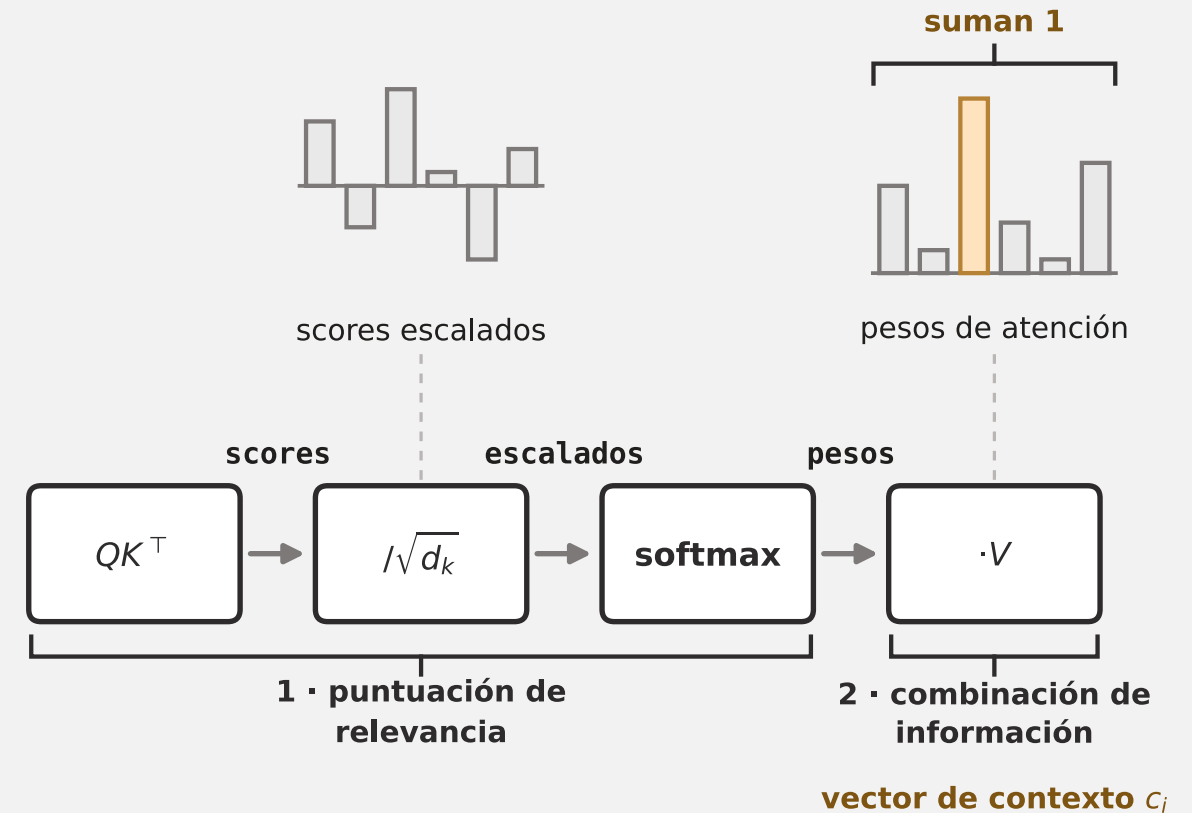
13

La fórmula, en cuatro pasos

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

- La atención de producto punto escalada, leída en cuatro pasos
- $QK^{\top}$: los scores sin normalizar, un producto punto de cada query con cada key
- $/\sqrt{d_k}$: los scores escalados, para no saturar el softmax
- softmax: los pesos de atención, no negativos y sumando 1 por fila
- $\cdot V$: el vector de contexto. Los tres primeros pasos son **puntuación de relevancia**; este es **combinación de información**, y son los dos pasos de los que está hecha la atención
- Se eligió producto punto y no atención aditiva por velocidad: es una multiplicación matricial optimizada

La salida es un promedio ponderado, no una elección



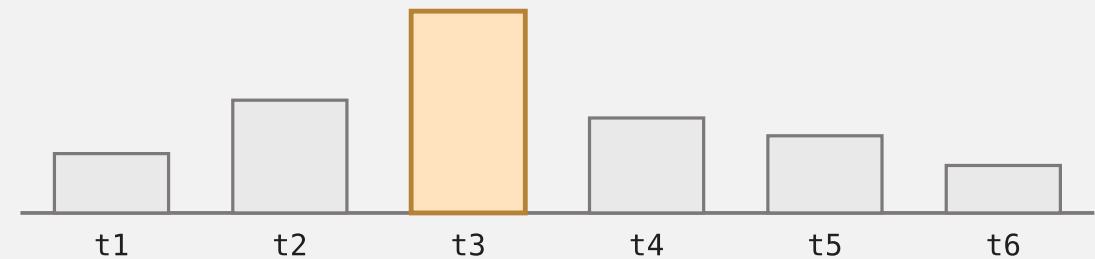
cada value entra con su peso; ninguno se descarta

Por qué se divide entre $\sqrt{d_k}$

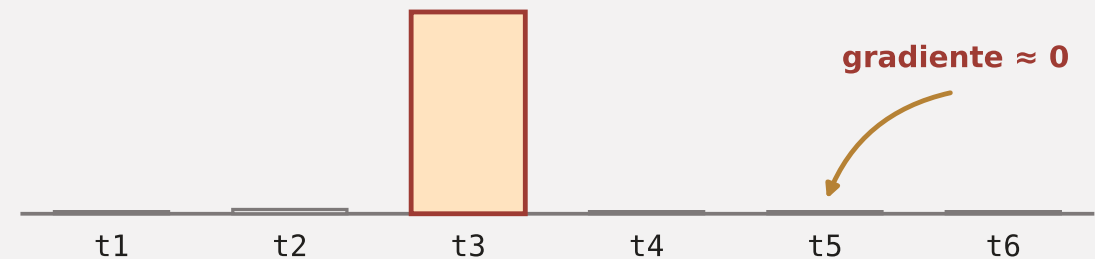
- Si las componentes de q y k son independientes, con media 0 y varianza 1, el producto punto tiene media 0 y varianza d_k
- Con d_k grande los productos punto crecen y empujan al softmax a regiones de gradientes extremadamente pequeños
- Dividir entre $\sqrt{d_k}$ devuelve la varianza a 1, que es donde el softmax discrimina sin saturarse
- Raschka lo dice desde el entrenamiento: al crecer los scores, el softmax se comporta como una función escalón
- Es una línea de código, y sin ella la arquitectura aprende muy lento o no aprende
- La Parte 5 del notebook lo verifica con $d = 4, 64$ y 256 : con 256 el softmax colapsa a one-hot

Sin $\sqrt{d_k}$ el softmax se satura y deja de discriminar

escalado



sin escalar, d grande



gradiente ≈ 0

la Parte 5 del notebook lo verifica con $d = 4, 64$ y 256

Máscara causal: $-\infty$ antes del softmax

- Restringe cada posición a las anteriores y a sí misma, y así preserva la propiedad autorregresiva
- Vaswani: poner a $-\infty$ las conexiones ilegales en la entrada del softmax; $e^{(-\infty)}$ tiende a 0
- La alternativa ingenua, enmascarar después del softmax y renormalizar, da el mismo resultado y cuesta más
- Sin máscara, el modelo vería el token que debe predecir y la factorización sería falsa

```
import numpy as np

def atencion(Q, K, V, causal=False):
    d_k = K.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)
    if causal:
        n = scores.shape[0]
        mascara = np.triu(np.ones((n, n), dtype=bool), k=1)
        scores = np.where(mascara, -np.inf, scores)
    pesos = softmax(scores, axis=-1)
    return pesos @ V, pesos
```

Tres tokens, a mano

- Con $Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$ y $d = 2$, los scores son $\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$
- Pesos causales ya escalados: fila 1 = $[1, 0, 0]$; fila 2 = $[0.330, 0.670, 0]$; fila 3 = $[0.333, 0.333, 0.333]$
- La fila de 'gato' sin escalar sería $[0.269, 0.731]$: ahí sí se ve qué hace el $\sqrt{2}$
- La fila de 'duerme' tiene scores $[1, 1, 1]$: su softmax es uniforme con y sin escalado. Ahí no hay nada que ver
- Salida del segundo token: $0.330 \cdot v_1 + 0.670 \cdot v_2 = [0.330, 1.340]$. Una suma ponderada, no una elección
- Los números del notebook coinciden con estos: por eso el papel y lápiz va primero

Ningún token mira hacia adelante

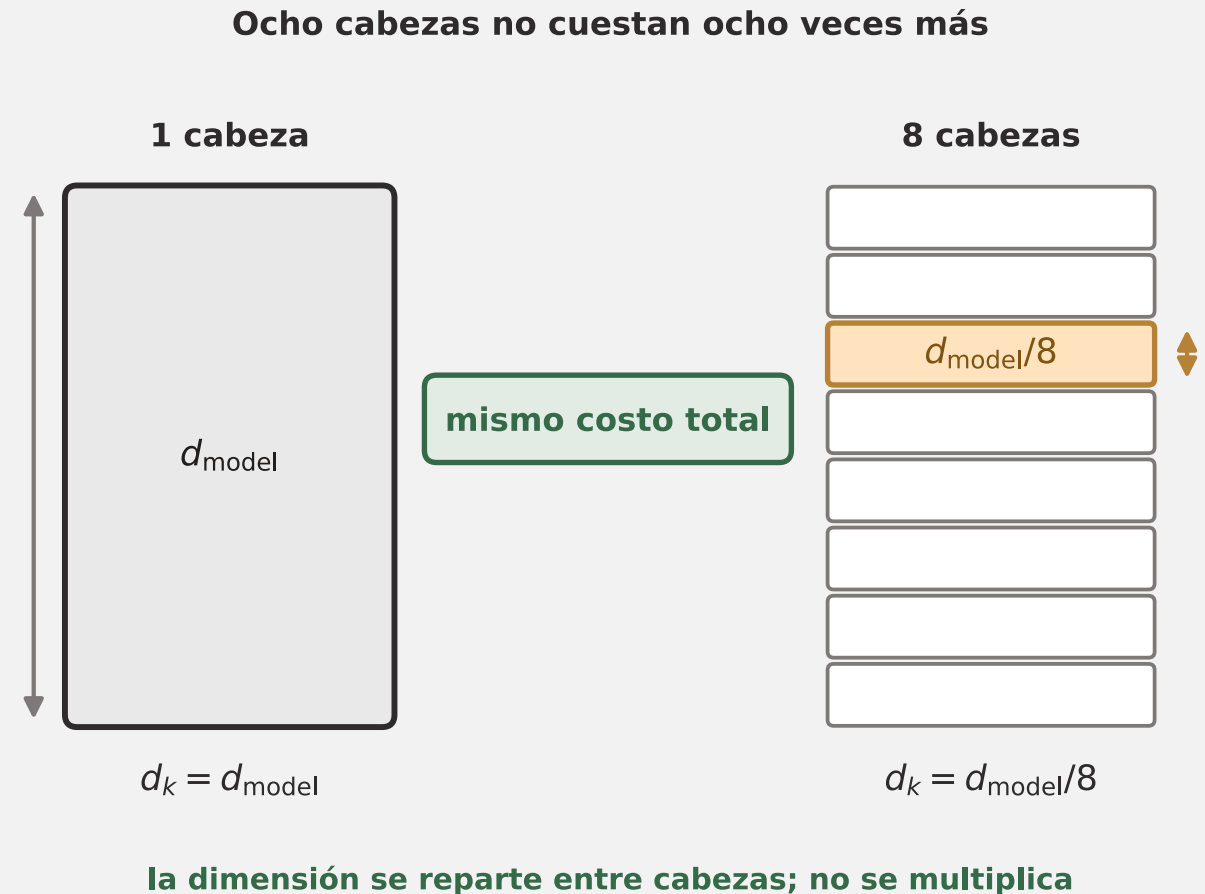
		key		
		el	gato	duerme
query	el	1.000	0	0
	gato	0.330	0.670	0
	duerme	0.333	0.333	0.333

sin escalar: 0.269 / 0.731

el triángulo superior es la máscara causal

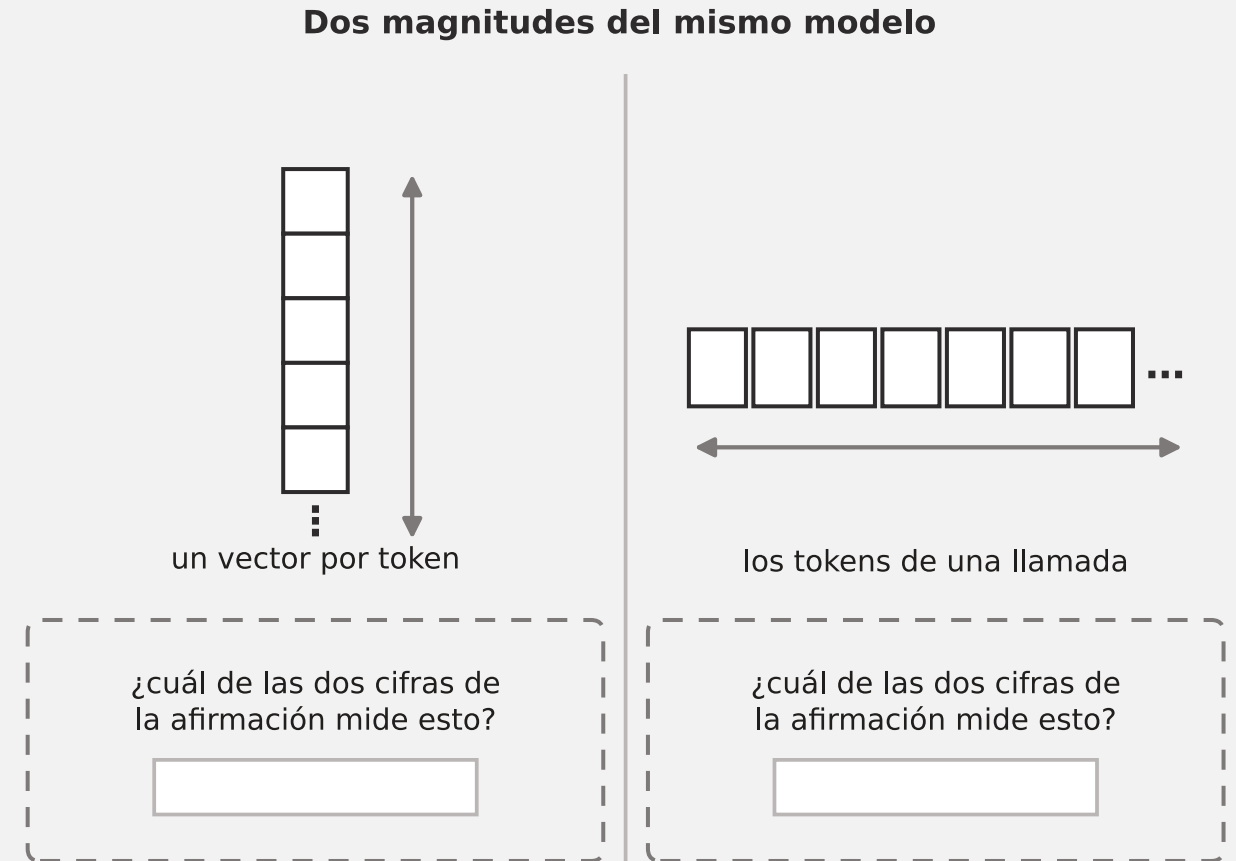
Multi-cabeza: h proyecciones distintas, el mismo costo

- En vez de una atención de dimensión d_{model} , h atenciones en paralelo con proyecciones lineales aprendidas distintas
- Las salidas se concatenan y se proyectan una vez más con W^O
- Motivo del paper: atender a información de distintos subespacios de representación en distintas posiciones; con una sola cabeza el promediado lo inhibe
- Con $d_k = d_v = d_{\text{model}}/h$ el costo total es similar al de una sola cabeza de dimensión completa (en el modelo base, $h = 8$ y $d_k = 64$)
- Más no es mejor: con cómputo constante, 1 cabeza da 24.9 BLEU —la métrica de calidad de traducción—, 8 dan 25.8 y 32 bajan a 25.4 (conjunto de desarrollo inglés→alemán, Vaswani 2017, Tabla 3)
- Sobre ‘cada cabeza aprende sintaxis’: el paper dice que muchas ‘parecen exhibir’ ese comportamiento. Se observa, no se programa



‘El transformer original tenía 65 M de parámetros y contexto 512’

- A. Correcto: 512 era la ventana de contexto del modelo base de 2017
- B. Los 65 M son correctos, pero 512 es d_{model} , la dimensión del modelo, no la ventana
- C. Las dos cifras están mal: el modelo base tenía 213 M de parámetros



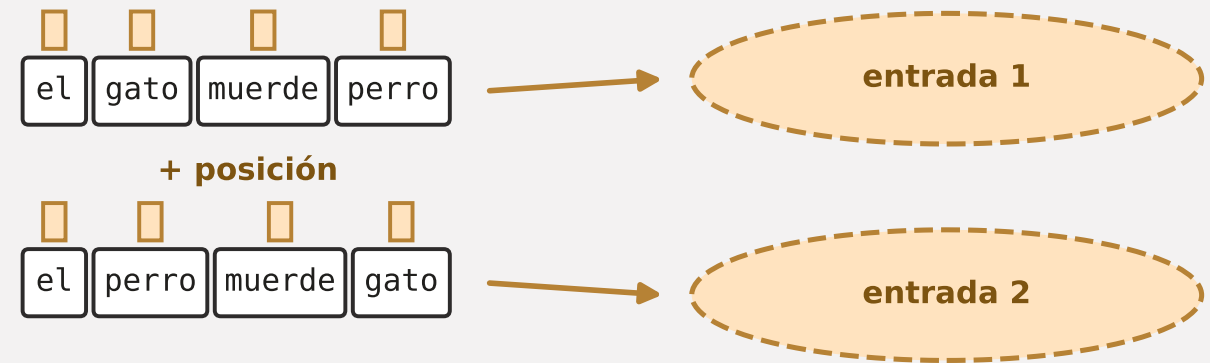
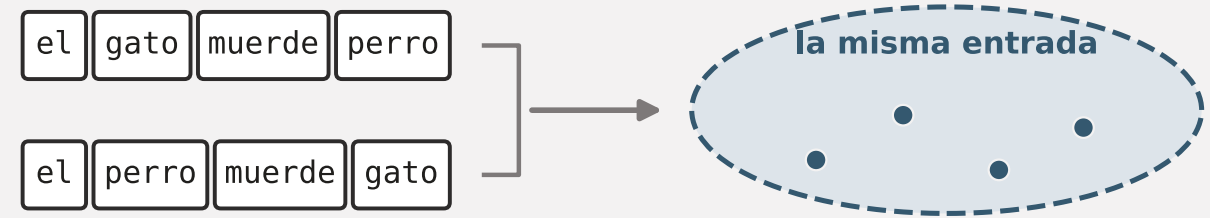
Posición, encoder y decoder

02

La atención es agnóstica a la posición

- El modelo no tiene recurrencia ni convolución, así que hay que inyectar la posición explícitamente
- La capa de embedding mapea siempre el mismo ID al mismo vector, esté donde esté en la secuencia
- Sin señal de posición, ‘el gato muerde al perro’ y ‘el perro muerde al gato’ son el mismo conjunto de vectores
- En las variantes absolutas la señal se suma al vector del token, y por eso tiene la misma dimensión d_{model}
- Dos absolutas —sinusoidal fija, la del paper, y aprendida, la de los modelos GPT— y una rotatoria (RoPE), que en vez de sumarse a la entrada rota queries y keys justo antes de puntuar
- Ablación del paper: aprendida y sinusoidal dan resultados casi idénticos. Lo que importa no es la fórmula, es que la señal esté

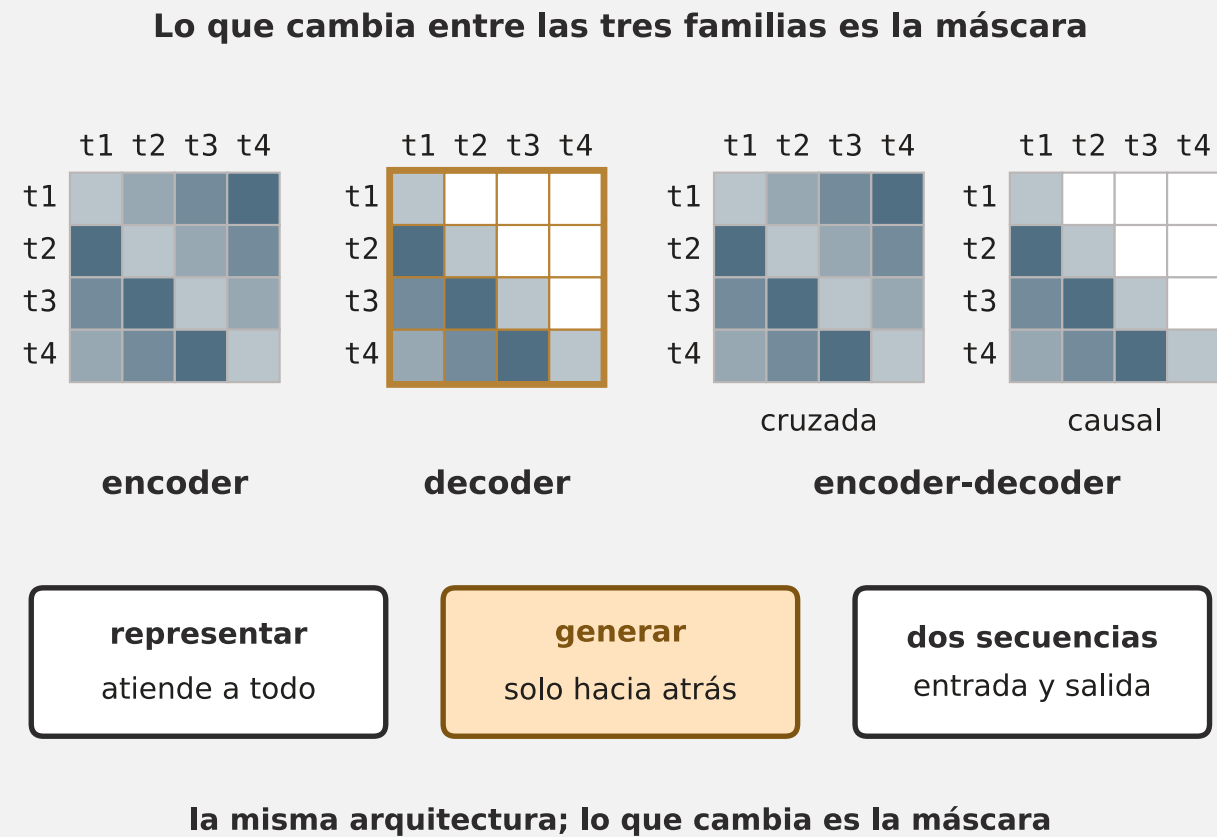
Sin señal de posición, el orden no existe



la señal de posición es lo que separa las dos frases

Encoder, decoder, y por qué los LLM son decoder-only

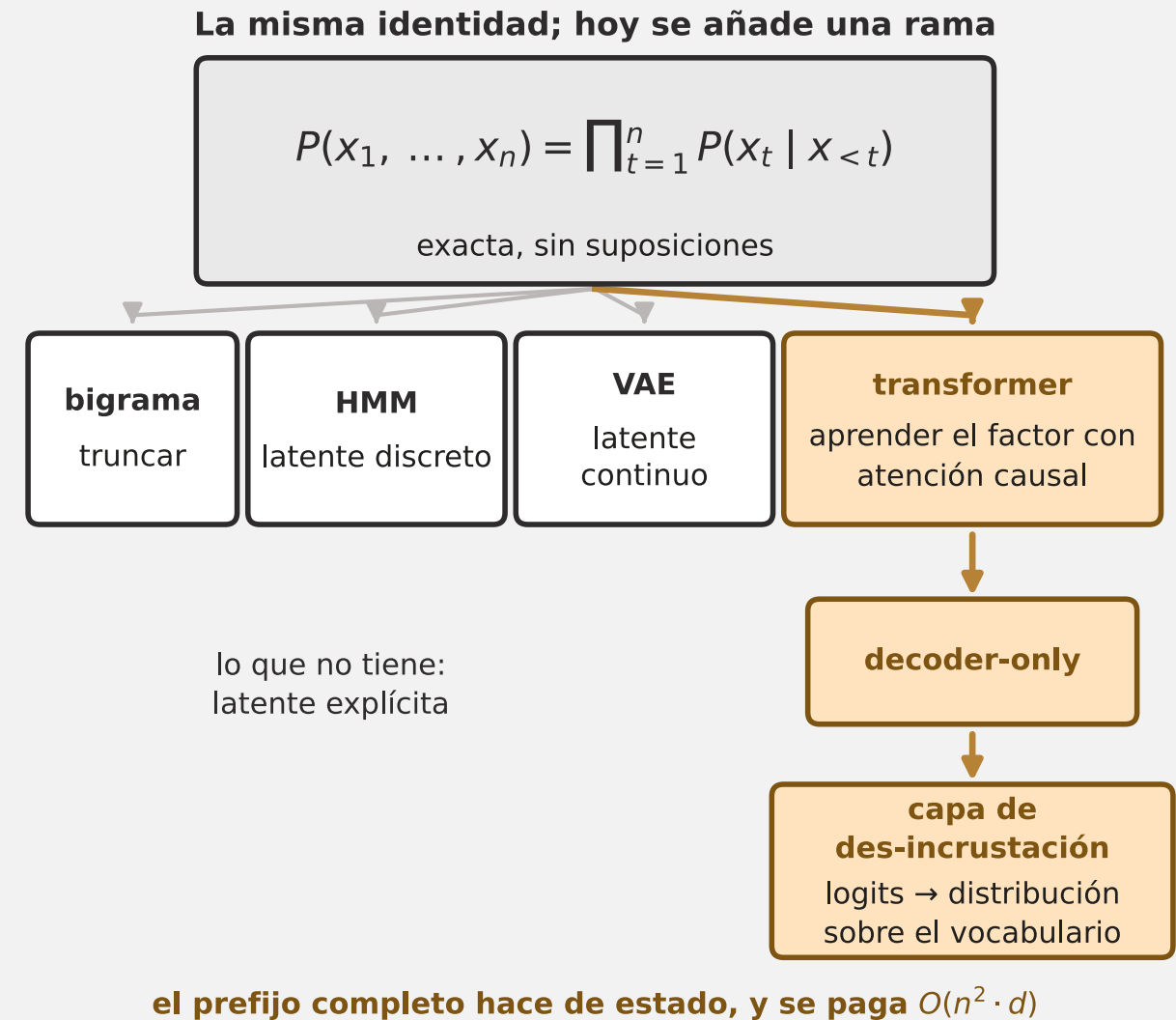
- Encoder: cada posición atiende a todas las de la capa anterior. Es un modelo de representación: clasifica y representa (BERT)
- Decoder: cada posición atiende hasta la actual, por la máscara causal. Sirve para generar (GPT)
- Encoder-decoder: añade una sub-capa donde las queries vienen del decoder y las keys y values de la salida del encoder (T5, BART)
- En modelado de lenguaje puro no hay una secuencia de entrada separada: el texto de entrada es el prefijo de la misma secuencia
- Quitar el encoder quita esa tercera sub-capa y quedan dos: auto-atención causal y red feed-forward. Es la arquitectura de los modelos generativos actuales, y el libro la describe así
- Falso amigo: el codificador del VAE es probabilístico sobre un latente continuo; el encoder del transformer es una pila de capas sobre tokens



Fuente: Vaswani et al. (2017), §3.1 p. 3, §3.2.3 p. 5 y §3.3 p. 5; Raschka (2024), cap. 2 p. 17 y §3.5 p. 74; Alammari & Grootendorst (2024), cap. 3, pp. 76 y 97, y §«Inside the Transformer Block» pp. 85–86; fuentes/notas-teoricas/s1-transformers-y-foundation-models.md, §3.6; Kingma & Welling (2013), §2.1, p. 3

Cómo el transformer implementa $P(x_t \mid x_{<t})$

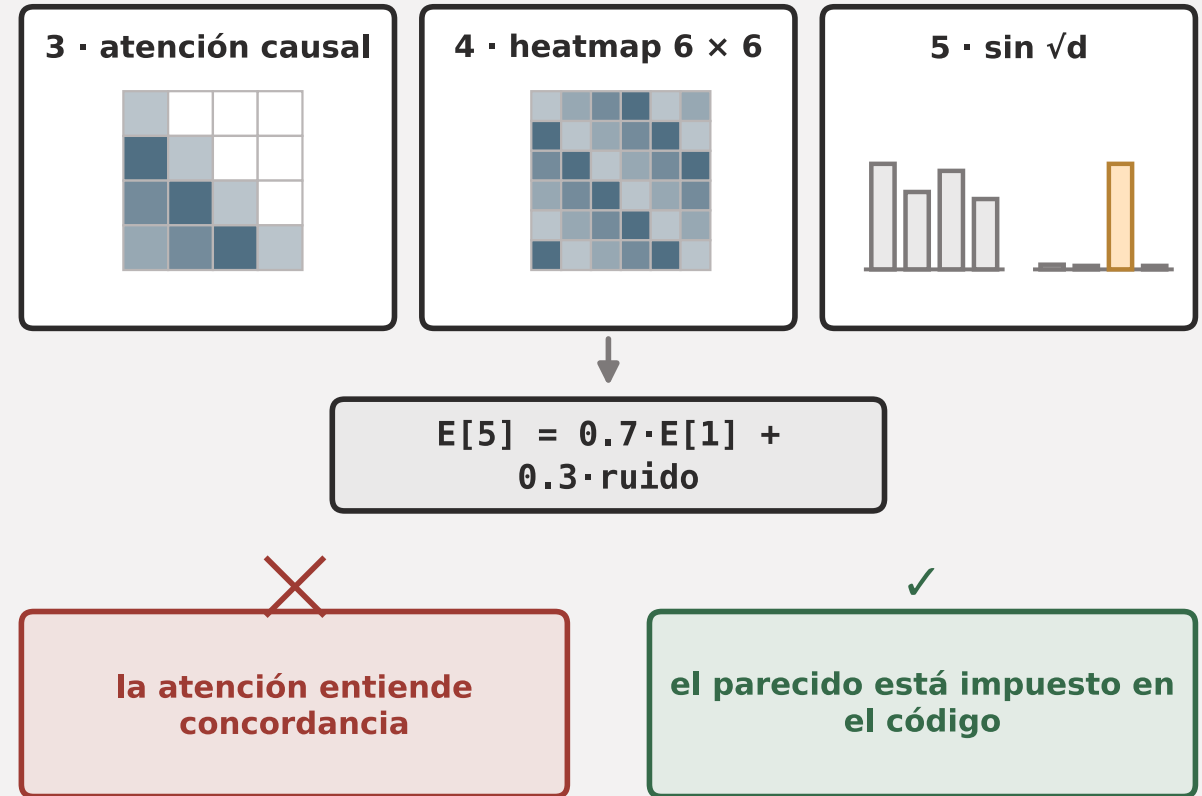
- La identidad $P(x_1, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_{<t})$ es exacta: todavía no es un modelo
- El bigrama trunca a un paso; el HMM mete un latente discreto; el VAE, uno continuo
- El transformer aprende el factor con una red que ve todo el prefijo: la alternativa que Bishop apuntaba en 2006
- La máscara causal obliga a respetar la factorización; la capa de des-incrustación proyecta el vector del último flujo al vocabulario y el softmax da la distribución
- El entrenamiento es auto-supervisado: pares de prefijo y siguiente token, de una ventana deslizando sobre el corpus
- Lo que no tiene: latente explícita. El contexto completo actúa como estado, y se paga $O(n^2 \cdot d)$



La actividad de hoy: atención en NumPy

- Parte 1: softmax estable. Parte 2: los tres tokens de recién, ahora en código
- Parte 3: atención causal con `np.triu` y $-\infty$. Verificar los ceros exactos encima de la diagonal
- Parte 4: heatmap de una mini-frase de seis tokens, usando los embeddings directamente como Q, K y V
- Ojo con la Parte 4: sin proyecciones aprendidas nada se aprende, y el parecido entre dos tokens está escrito en el código
- El heatmap ilustra el mecanismo; no es evidencia de que la atención entienda concordancia
- Parte 5: el experimento sin $\sqrt{d}$ con $d = 4, 64$ y 256 . Corre con numpy y matplotlib, sin ninguna clave de API

El heatmap ilustra el mecanismo; no demuestra comprensión



la simplificación es de la actividad, no del mecanismo